feature/audit-deep-read — Branch Report

Project: Smart Expense & Budget Tracker · Branch: `feature/audit-deep-read` · 12 commits ahead of `master` · Generated 2026-04-14

1. What this branch actually is

This branch is **not a feature**. It is a **governance and infrastructure retrofit** layered on top of the working prototype that already landed on `master`. Application code is untouched. Every commit is either documentation or Claude Code tooling.

> **One-line summary:** It turns a single-dev prototype into a project that can survive a second contributor, an AI assistant, and a code review — without rewriting a line of app code.

Composition of the 12 commits

#	Commit	Kind
1	docs: governance, API, security, testing, migration docs	docs
2	docs: database design, split frontend/backend docs	docs
3	docs: verify against source, fix factual errors, testing guides	docs
4	docs: ticket inventory, testing checklist, roadmap	docs
5	docs: restructure into department folders for SDLC	docs
6	docs: cross-functional contracts, department workspaces	docs
7	docs: ADR scaffolding + three backfilled decisions	docs
8	docs(governance): update CLAUDE.md to reflect restructure	governance
9	chore(infra): .claude/ project init for SDLC enforcement	infra
10	chore(infra): project-tailored statusline	infra
11	chore(infra): adopt 6 slash commands	infra
12	chore(infra): restructure .claude/ to folder-per-skill	infra

Totals: 152 files changed, +28,255 / −261 lines. Zero lines of `backend/` or `frontend/src/` touched.

2. Why this work was done (the reasoning)

The problem before this branch

The `master` branch held a working React + Flask prototype (~21k LOC), but:

- Documentation existed as a **flat pile** in `Documents/` — no ownership, no lifecycle, no hand-off structure.

- There was **no constitution** (CLAUDE.md existed but was thin). No rules to prevent a refactor-while-fixing-a-bug drift.
- Six financial formulas (`budget_remaining`, `saved`, `usage_percent`, alert thresholds, `highest_category`, `avg_daily_spending`) had **no gate** preventing accidental modification.
- Claude Code was running with **default permissions** — no guardrails, no project-specific knowledge, no reviewable decisions trail.
- There was **no ticket inventory**, **no test checklist**, **no release runbook**. Every bug fix relied on memory.

The thesis

> A one-person prototype that treats itself as a real project from day one is cheaper to scale later than a prototype that tries to add process after it has grown. The cost of writing `CLAUDE.md` Rule 7 ("No raw SQL") today is five minutes. The cost of ripping raw SQL out of 30 routes in month nine is a sprint.

The explicit goals of the branch

1. **Make the docs survive a new contributor.** A developer joining next week should be able to read `Documents/Reference/PROJECT_CONTEXT.md`, then the relevant department folder, and ship a ticket without asking.
2. **Protect business logic mechanically, not by trust.** Formula changes must be flagged `[BIZ-QC-NEEDED]` and pass a reviewer agent.
3. **Turn Claude Code from a freelancer into a junior team member** — one that reads the same rules, follows the same workflow, and produces reviewable output.
4. **Replace memory with documents.** Anything you'd tell a new hire verbally gets written down once.

3. What actually shipped

3.1 Documentation restructure

The flat `Documents/` folder was reorganized into **nine SDLC departments**. Each department has a `README.md` and owns a specific concern:

```
Documents/
├── Reference/    ← what the system IS (PROJECT_CONTEXT, SRS, API, DB, stack)
├── Process/      ← how work moves (GitWorkFlow, DoR, DoD, RACI, standards)
├── Product/      ← what & why we build (roadmap, personas)
├── Project/      ← execution tracking (25-ticket inventory, sprints)
├── Engineering/  ← how it's built (Backend/, Frontend/, Architecture/ADR/)
├── QA/           ← how we verify (strategy + 70-item checklist)
├── DevOps/       ← how it ships (setup, deployment, SLOs, runbook)
├── Security/     ← threat model, incident response, compliance
└── Design/       ← placeholder (no designer yet)
```

New documents that did not exist before this branch:

- `Process/RACI.md` — who is Responsible / Accountable / Consulted / Informed for each workflow (W1–W8).
- `Process/DefinitionOfReady.md` + `DefinitionOfDone.md` — objective gates for ticket pickup and PR merge.
- `Project/ticket-inventory.md` — 25 seeded tickets (BUG-/FEAT-/INFRA-/TEST-/DOCS-).
- `QA/TestingChecklist.md` — 70 enumerated test cases, one tick per row.
- `Engineering/Architecture/ADR/` — three backfilled decisions (SQLite→MySQL, JWT-in-localStorage, no global state).
- `Security/SecurityAndThreatModel.md` + `IncidentResponse.md`.
- `DevOps/ReleaseRunbook.md` + `MigrationPlan.md` + `SLOs.md`.

3.2 The CLAUDE.md constitution

`CLAUDE.md` is the one file every contributor — human or AI — must obey. It codifies **eight rules**:

1. **Read before write.** Read the file and its imports before modifying.
2. **Protected business logic.** The six financial formulas require `[BIZ-QC-NEEDED]` + sign-off.
3. **Protected branches.** No direct commits to `master`, no force-push.
4. **Never rename legacy files.** Preserves existing URLs and muscle memory.
5. **Bug fix ≠ refactor.** Fix what the ticket says. Extra changes require a new ticket.
6. **One ticket per branch.** No "while I was here" bundling.
7. **No raw SQL — ORM only.** All DB access through SQLAlchemy.
8. **If unsure → escalate, don't guess.** Wrong guess on finance math costs more than delay.

3.3 The `.claude/` infrastructure

This is the **enforcement layer** — the machinery that turns the rules above from aspiration into automation:

```
.claude/
├── settings.json              ← permissions (allow/deny/ask) + hooks + statusline
├── hooks/
│   └── check-branch.py        ← blocks commits on master/main (Rule 3)
├── agents/
│   ├── biz-qc-reviewer.md     ← re-derives the 6 formulas before approving
│   ├── scope-checker.md       ← flags Rule 5/6 violations
│   ├── orm-only-checker.md    ← enforces Rule 7 (no raw SQL)
│   ├── qa-coverage-checker.md ← verifies tests match code changes
│   └── security-reviewer.md   ← maps diffs to SEC-* threat items
└── skills/                    ← 10 slash-command workflows
    ├── check-ready/  check-done/  commit/  done/
    ├── lint-fix/     arch-doc/    update-docs/
    ├── weekly/       release-preflight/  statusline-setup/
```

4. How it works (the machinery)

4.1 The four layers and what each does

Layer	Lives in	Purpose	When it runs
Rules	`CLAUDE.md` + `Documents/`	Declare the law. Canonical source of truth.	Read by every contributor at session start.
Hooks	`.claude/hooks/`	Hard gates. Refuse the action outright.	Before a tool call (e.g., before `git commit`).
Agents	`.claude/agents/`	Advisory reviewers. Return PASS/FAIL with reasoning.	On-demand, or automatically when the main Claude sees a matching diff.
Skills	`.claude/skills/`	Workflows — macros that chain reads, checks, and writes.	Invoked by the user via slash commands (e.g., `/commit`).

4.2 Walk-through: fixing one bug end-to-end

Scenario: fixing **BUG-1** (savings_goal reset) from the ticket inventory.

```
1. /check-ready BUG-1
   → skill reads the ticket, checks DoR gates, says READY or NOT READY.

2. Branch: git checkout -b bugfix/BUG-1-savings-goal-reset
   → check-branch.py hook allows this (not master).

3. Fix the code. Write a regression test.
   → main Claude obeys Rule 5 (fix only what's in the ticket).

4. Automatic agent review (main Claude spawns these):
   → scope-checker    : "did we only touch BUG-1's files?"   CLEAN
   → biz-qc-reviewer  : touches saved formula → PASS, [BIZ-QC-NEEDED] present
   → orm-only-checker : no raw SQL in diff → PASS
   → qa-coverage      : test added & checklist row ticked → PASS
   → security-reviewer: no SEC-* items touched → PASS

5. /commit
   → skill stages files, composes message per GitWorkFlow.md, commits.
   → check-branch.py hook: not master, commit allowed.

6. /done
   → runs lint-fix, updates docs, ticks checklist row,
      writes productivity report, verifies DoD gates 1-7.

7. Push & open PR. Human reviewer signs off. Merge. Delete branch.
```

4.3 How an agent actually gets invoked

An agent file like `biz-qc-reviewer.md` is **not code** — it is a persona definition with frontmatter (`name`, `tools`, `model`) and a system prompt. When the main Claude decides to invoke it, a fresh Claude

instance boots with that system prompt, limited to the listed tools, runs the task, and returns one structured report. The subagent has no memory of prior sessions. The verdict is advisory; a human always holds final approval.

> **Why this design works:** context isolation. A single mega-reviewer trying to check business logic, scope, ORM compliance, tests, and security would forget things. Five narrow agents, each with one job, give reliable answers and can run in parallel.

4.4 Purpose of each individual piece

Hooks — `check-branch.py`

Runs before any `git commit`. If the current branch is `master` or `main`, the commit is refused. This is Rule 3 as a mechanical gate, not a norm.

Agents

Agent	Enforces	Triggers on
`biz-qc-reviewer`	Rule 2 — six protected formulas	diffs in `backend/routes/{budget,reports,expenses}.py`, `models.py`
`scope-checker`	Rules 5 & 6 — no refactor, one ticket per branch	any PR
`orm-only-checker`	Rule 7 — no raw SQL	diffs in `backend/routes/`, `backend/models.py`
`qa-coverage-checker`	DoD — tests + checklist ticked	any PR touching code
`security-reviewer`	Threat model SEC-* controls	diffs in `app.py`, `routes/auth.py`, new input fields

Skills — the ten slash commands

Skill	What it does
`/check-ready`	Walks DoR gates against a ticket. Reports READY or NOT READY.
`/check-done`	Walks DoD gates against the current branch before PR.
`/commit`	Stages, writes conventional commit message, auto-detects `[BIZ-QC-NEEDED]`.
`/done`	End-of-session wrap: lint, tests, doc updates, productivity report, final commit.
`/lint-fix`	Runs flake8/black + ESLint on touched files only (no speculative refactors).
`/arch-doc`	Creates or updates an ADR from the template.
`/update-docs`	Reads recent code changes and updates the affected department docs.
`/weekly`	Generates a weekly summary of master commits — categorized, with standup draft.
`/release-preflight`	Runs the pre-flight checklist from the release runbook.
`/statusline-setup`	Configures the two-row statusline with `[BIZ-QC]` indicator.

5. What this branch is NOT

- **Not a feature.** Zero user-visible behavior change.
- **Not a refactor.** No code moved or renamed.
- **Not a release.** The prototype on `master` is the product; this branch is the scaffolding around it.
- **Not merged yet.** 12 commits ahead of `master`, unpushed at time of writing.
- **Not complete.** Three items deliberately left uncommitted (CORS widening, `PROJECT_CONTEXT.md` relocation, `package-lock.json`) — they need an owner decision.

6. Known gaps and recommended follow-ups

Gap	Impact	When to fix
No CI wired (GitHub Actions)	Hooks are local-only; a collaborator bypassing `.claude/` is not blocked.	On second contributor.
70 checklist rows, 0 tests written	DoD "test suite green" is aspirational today.	Add one test per bug fix. Do not backfill all 70 at once.
No `.env.example`	Onboarding friction.	Before next contributor joins.
No `/resume` skill	Session pickup relies on memory + manual re-read.	If session-hopping becomes frequent.
Scope-checker references "Out-of-scope" bullets not present in ticket inventory	Minor agent drift.	2-minute fix: update agent to infer from Notes column.
Alembic migration config missing	MigrationPlan.md describes a process that cannot yet run.	Before SQLite→MySQL cutover.
No dependency auditing (Dependabot / `pip-audit` / `npm audit`)	Security doc prescribes it; nothing runs.	Before first public release.

7. The honest assessment

> The scaffolding is ~90% of what a solo-dev-with-Claude project needs. The remaining 10% is the stuff that only pays off with a second contributor or real user traffic — CI, Dependabot, a migration runner. Adding them now would be premature. The right next step is not more infrastructure; it is **using the infrastructure** — picking up BUG-1, running the full workflow, and seeing where it breaks in practice.

8. Recommended next actions (ranked)

1. Decide on the three uncommitted items (CORS, `PROJECT_CONTEXT.md` move, `package-lock.json`).
2. Push `feature/audit-deep-read`, open PR, merge. Dead branches rot.
3. Fix the scope-checker "Out-of-scope" reference.

4. Pick up BUG-1 (savings_goal reset). Run the full `/check-ready` → fix → `/done` loop end-to-end. Learn where the workflow stutters.
5. Wire GitHub Actions only after step 4 reveals which checks are actually worth running in CI.
